УДК 004.45

СРАВНИТЕЛЬНЫЙ АНАЛИЗ ФОРМАТОВ РАСПРОСТРАНЕНИЯ ПО RPM, FLATPAK, APPIMAGE В СРЕДЕ ALT WORKSTATION 10.4

Бережная П.А., Мирчевски К.Д.

Резюме. В статье представлено экспериментальное сравнение трёх форматов распространения программного обеспечения в Linux: RPM, Flatpak и AppImage. Исследование выполнено на платформе ALT Workstation 10.4, активно внедряемой в корпоративном и государственном секторах России. Актуальность работы обусловлена переходом на отечественные операционные системы и необходимостью выбора оптимального формата доставки приложений.

Научная новизна заключается в том, что впервые проведено комплексное сравнение указанных форматов именно на ALT Workstation 10.4 по трём группам критериев: производительность, безопасность и сопровождение.

Цель исследования – экспериментально сравнить RPM, Flatpak и AppImage в среде ALT Workstation 10.4 и выработать практические рекомендации для корпоративных сценариев.

Методология включала измерение времени запуска (hyperfine), потребления RSS-памяти, системных вызовов (strace), файлового доступа (lsof), сетевой активности, модели прав, проверки целостности и занимаемого дискового пространства.

Эксперименты показали, что RPM обеспечивает наименьшие накладные расходы и лучшее использование ресурсов благодаря тесной интеграции с ОС, Flatpak предоставляет наиболее надёжную изоляцию и управление правами, но требует больше памяти и генерирует больше системных вызовов, а AppImage демонстрирует вариативное поведение: одни приложения запускаются быстро, другие – заметно медленнее из-за автономной упаковки.

Полученные результаты могут помочь при выборе формата распространения ПО для корпоративных Linux-сред на базе ALT Linux. Ограничения связаны с виртуальной средой и режимом AF_PACKET. Перспективы – тестирование на Astra Linux и РЕД ОС.

Ключевые слова: RPM, Flatpak, AppImage, ALT Workstation, Linux, сравнительный анализ

COMPARATIVE ANALYSIS OF RPM, FLATPAK, AND APPIMAGE DISTRIBUTION FORMATS IN THE ALT WORKSTATION 10.4 ENVIRONMENT

Berezhnaya P.A., Mirchevski K.D.

Summary. The purpose of this study is to conduct an experimental comparison of three software distribution formats – RPM, Flatpak, and AppImage –within the ALT Workstation 10.4 environment.

The scientific novelty lies in the first systematic evaluation of these formats on a domestic operating system, focusing on performance, security, and maintainability metrics, thereby providing quantitative data previously unavailable for this specific platform.

The research measured application startup time, RSS memory consumption, system call activity, and disk space usage across identical virtual machines. The results demonstrate that RPM packages ensure the tightest OS integration with minimal resource overhead. Flatpak offers the most robust sandbox isolation and explicit permission management, albeit with higher memory consumption and system call frequency. AppImage provides high portability but lacks centralized maintenance and built-in isolation. Ultimately, RPM is recommended for resource-constrained corporate environments, Flatpak for security-critical scenarios, and AppImage for portable applications.

The obtained results can help when choosing a software distribution format for corporate Linux environments based on ALT Linux. The limitations are related to the

virtual environment and the AF_PACKET mode. Prospects include testing on Astra Linux and RED OS.

Keywords: RPM, Flatpak, AppImage, ALT Workstation, Linux, comparative analysis

Введение

Переход на отечественные операционные системы (далее – ОС), такие как ALT Workstation, Astra Linux, РЕД ОС, ставит задачу выбора оптимального формата распространения программного обеспечения (далее – ПО). Выбранная нами ОС ALT Workstation, разработанная российской компанией «Базальт СПО», традиционно использует системный пакетный формат RPM и менеджер APT, обеспечивая высокую интеграцию и управляемость, однако в последние годы большее распространение в экосистеме Linux получают универсальные форматы доставки приложений, такие как Flatpak и AppImage. Они позиционируются как решения, которые позволяют разработчикам обходить проблему зависимостей, а пользователям – получать свежие программные версии без привязки к конкретному дистрибутиву.

Несмотря на наличие ряда работ, сравнивающих форматы контейнеризации или пакетные менеджеры, комплексное исследование, объединяющее оценку производительности, безопасности и удобства сопровождения для этих трех форматов именно на платформе ALT Workstation 10.4, до этого времени не проводилось, следовательно, отсутствуют количественные данные о том, насколько сильно отличаются время запуска, потребление памяти, системная нагрузка, уровень изоляции, дисковые затраты при использовании одного и того же приложения, установленного через RPM, Flatpak и AppImage. Практическая ценность исследования заключается в получении экспериментальных данных о работе RPM, Flatpak и AppImage в среде ALT Workstation 10.4 Эти данные могут использоваться при выборе формата распространения приложений для рабочих станций и корпоративных Linux-систем.

Литературный обзор

Актуальность сравнительного анализа форматов распространения ПО в Linux-средах обусловлена необходимостью выбора оптимального способа доставки приложений на рабочих станциях, особенно в условиях перехода на отечественные ОС, например, ALT Workstation 10.4. В литературе и технической документации рассматриваются три основных подхода: классические системные пакеты (RPM), изолированные форматы с рантаймом (Flatpak) и переносимые исполняемые файлы (AppImage) [16].

RPM (RPM Package Manager) является традиционным форматом пакетов, используемый в ALT Linux, Red Hat и производных дистрибутивах. Согласно официальной документации ALT Workstation 10.4 и руководству rpm(8), пакет представляет собой архив cpio с метаданными, такими как версия, архитектура, зависимости, контрольные суммы [1] [13]. Проверка целостности выполняется командой rpm -V, которая сравнивает атрибуты установленных файлов с записью в локальной базе данных [5]. Однако RPM не предоставляет встроенной изоляции приложений и безопасность обеспечивается лишь правами пользователя и дополнительными модулями (AppArmor, SELinux). Поэтому можно сделать вывод, что основным преимуществом RPM являются минимальные накладные расходы и полная интеграция с системой.

Flatpak является современной системой изоляции и распространения настольных приложений. Как описано в документации Flatpak, каждое приложение запускается в песочнице (Linux namespaces, seccomp), изолированной от хостовой ОС, а доступ к файлам, сети, графическим сокетам и устройствам предоставляется через декларативные разрешения и порталы (xdg-desktop-portal) [6] [14] [17]. Обновления атомарны благодаря использованию OSTree – системы версионирования файлов с дедупликацией [2]. Таким образом, Flatpak обеспечивает высокий уровень безопасности, но требует дополнительных рантаймов и дискового пространства.

AppImage является форматом переносимых приложений, не требующих установки. По данным официального сайта и вики ALT Linux, AppImage

представляет собой самоподключаемый образ (через FUSE), содержащий приложение и все его зависимости в одном исполняемом файле [4] [3]. Пользователь может запустить AppImage после установки прав на выполнение без прав root. Однако формат не имеет встроенной модели разрешений и песочницы, его работоспособность зависит от версии FUSE (конфликты между FUSE 2 и FUSE 3) [10]. Проверка целостности возможна только вручную (SHA256).

На основании проведенного анализа литературы сформулированы следующие гипотезы:

H1: Формат распространения оказывает влияние на производительность запуска, потребление памяти и системную активность приложений.

H2: Flatpak обеспечивает более выраженные механизмы изоляции и контроля разрешений по сравнению с RPM и AppImage.

H3: AppImage обеспечивает наибольшую автономность и переносимость приложений, но не предоставляет встроенных механизмов централизованного сопровождения и изоляции.

Методы исследования

Для сравнительного эксперимента использовались отдельные виртуальные машины с идентичными параметрами: ОС ALT Workstation 10.4 x86_64 с 4096 МБ оперативной памяти. Все тесты выполнялись от имени непривилегированного пользователя user, root/sudo применялись при необходимости для установки пакетов, очистки кэша, системных проверок и трассировок.

Для измерений использовались hyperfine, strace, lsof, ps, awk, sha256sum, rpm, flatpak, ostree и du. Hyperfine применялся для автоматизированного многократного измерения времени запуска и расчета среднего значения и стандартного отклонения [11]. Для обеспечения воспроизводимости эксперимента измерялось время выполнения команды запуска приложения без инициализации графического интерфейса, что позволило минимизировать

влияние пользовательской среды и графической подсистемы на результаты измерений.

В набор были включены приложения различных классов: мультимедийный проигрыватель (VLC), графические редакторы (GIMP, Inkscape), офисный пакет (LibreOffice) и система трехмерного моделирования (Blender), что позволило оценить поведение форматов распространения для различных типов пользовательского программного обеспечения (табл. 1)

Таблица 1. Тестовые приложения

Приложение	RPM	Flatpak	AppImage / portable
VLC	vlc	org.videolan.VLC	VLC_media_player-3.0.11.1-x86_64.AppImage
LibreOffice	LibreOffice-still	org.libreoffice.LibreOffice	LibreOffice-still.basic-x86_64.AppImage
GIMP	gimp	org.gimp.GIMP	GIMP-2.10.38.AppImage / AppRun
Inkscape	inkscape	org.inkscape.Inkscape	Inkscape-9dee831-x86_64.AppImage
Blender	blender	org.blender.Blender	Blender portable build

Ход исследования. Результаты производительности

Время выполнения команды запуска приложения в консольном режиме без инициализации графического интерфейса. Для измерения времени запуска использовалась утилита Hyperfine с двумя запусками: холодным, который выполнялся после очистки файлового кэша ОС, и теплый – без очистки кэша. Для тестов применялась команда вида hyperfine --runs N "<application> --version", где <application> обозначает исполняемый файл тестируемого приложения. Для Blender portable измерялась версия portable-сборки, которая использует иной механизм запуска по сравнению с RPM-пакетом (табл. 2).

Таблица 2. Время запуска: среднее ± σ

Формат	Приложение	Холодный запуск, с	Теплый запуск, с
RPM	Blender	0.220 ± 0.042	0.209 ± 0.012
RPM	VLC	0.008 ± 0.002	0.007 ± 0.001
RPM	GIMP	0.012 ± 0.002	0.025 ± 0.007

RPM	LibreOffice	0.351 ± 0.031	0.316 ± 0.018
RPM	Inkscape	0.139 ± 0.015	0.162 ± 0.020
Flatpak	Blender	3.717 ± 0.772	0.286 ± 0.016
Flatpak	VLC	0.644 ± 0.017	0.184 ± 0.029
Flatpak	GIMP	1.149 ± 0.041	0.237 ± 0.024
Flatpak	LibreOffice	2.079 ± 0.151	0.431 ± 0.029
Flatpak	Inkscape	1.895 ± 0.117	0.370 ± 0.051
AppImage	Blender	0.044 ± 0.005	0.042 ± 0.005
AppImage	VLC	3.027 ± 0.525	2.607 ± 0.355
AppImage	GIMP	0.121 ± 0.015	0.108 ± 0.015
AppImage	LibreOffice	2.965 ± 0.440	2.643 ± 0.338
AppImage	Inkscape	2.991 ± 0.438	1.475 ± 0.243

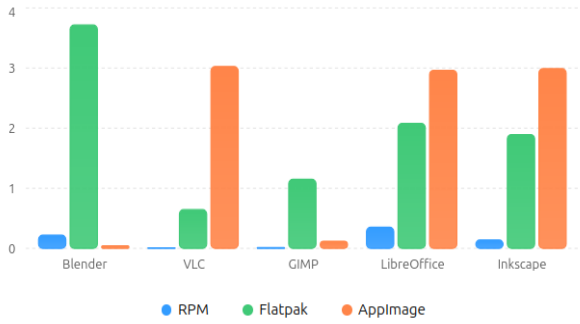


Рис.1. Среднее время выполнения команды заупска приложения в форматах RPM, Flatpak, AppImage

Результаты измерений показали, что в большинстве тестов RPM-пакеты запускались быстрее других рассматриваемых форматов, поскольку они используют системные библиотеки и не требуют дополнительной подготовки среды выполнения. Для Flatpak увеличение времени запуска наблюдалось практически во всех тестах, что обусловлено необходимостью инициализации runtime-компонентов и механизмов изоляции. В случае AppImage результаты зависели от конкретного приложения: это может быть связано с особенностями portable-сборок, включающих собственные зависимости и требующих дополнительной подготовки среды выполнения.

Потребление оперативной памяти

Для всех форматов приложение запускалось в фоновом режиме командой вида <application> &, после чего выполнялась задержка для завершения инициализации процесса. Размер RSS-памяти определялся командой ps -o rss= -p <PID>, где <PID> – идентификатор процесса приложения. Flatpak запуск осуществлялся через flatpak run <application-id>, а для AppImage через соответствующий исполняемый файл AppImage. Для Flatpak искался процесс внутри sandbox, для AppImage учитывались AppRun, FUSE и loader-процессы.

RSS фиксировался спустя 10 секунд после запуска приложения, что обеспечивало единообразные условия измерения. Показатель RSS не отражает фактическое распределение разделяемых страниц памяти между процессами и используется как приближенная характеристика потребления памяти, поэтому используется как практический показатель потребления памяти процесса, а не как абсолютная стоимость приложения в памяти. LibreOffice AppImage был исключен из итогового RSS-сравнения из-за невозможности стабильной фиксации целевого процесса в ходе эксперимента (табл. 3).

Таблица 3. Потребление оперативной памяти

Формат	Приложение	RSS, МБ
RPM	Blender	232.47
RPM	VLC	18.29
RPM	GIMP	41.35
RPM	LibreOffice	6.72
RPM	Inkscape	131.39
Flatpak	Blender	776.51
Flatpak	VLC	96.75
Flatpak	GIMP	179.36
Flatpak	LibreOffice	156.77
Flatpak	Inkscape	136.40
AppImage	Blender	520.03
AppImage	VLC	89.94
AppImage	GIMP	109.52

AppImage	LibreOffice	excluded
AppImage	Inkscape	140.73

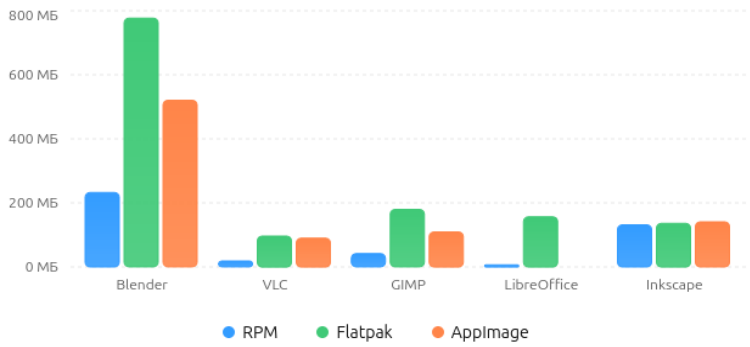


Рис.2. Сравнение потребления RSS-памяти приложений в форматах RPM, Flatpak и AppImage

Измерения RSS-памяти показали, что в большинстве проведённых тестов RPM-пакеты потребляли меньше оперативной памяти, чем другие рассматриваемые форматы, что связано с использованием системных библиотек и более тесной интеграцией приложений с операционной системой. Наибольшие значения RSS наблюдались у Flatpak-приложений, так как при запуске дополнительно задействуются runtime-компоненты и механизмы изоляции, которым также требуются ресурсы оперативной памяти. Для AppImage полученные значения чаще всего находились между RPM и Flatpak, поскольку они не используют sandbox-изоляцию, но при этом имеют встроенные зависимости, входящие в состав portable-пакета: по этой причине объем потребляемой памяти обычно оказывался выше, чем у RPM-версий тех же программ.

Результаты безопасности и sandbox-изоляции. Syscall analysis через strace

Для анализа системных вызовов использовался strace: сначала собиралась сводная статистика, затем детальный лог по openat, access, execve, socket и connect [15]. Для Flatpak часть трассировок выполнялась с sudo из-за особенностей bubblewrap и namespace-механизмов, это указано как ограничение единообразия условий [7]. Результаты следует интерпретировать как

качественное сравнение поведения приложений, а не абсолютное сравнение количества syscall. Для исследования взаимодействия приложений с операционной системой использовалась команда strace -f -c <application>, ключ -f обеспечивал трассировку дочерних процессов, а ключ -c формировал сводную статистику системных вызовов (табл. 4).

Таблица 4. Сводный strace-анализ

Формат	APP	Syscalls	Errors	openat	socket	connect
RPM	VLC	1122	56	122	0	0
RPM	GIMP	1048	57	142	0	0
RPM	Inkscape	3755	512	372	5	5
RPM	LibreOffice	4631	853	1049	4	2
Flatpak	VLC	25489	17035	788	32	17
Flatpak	GIMP	26288	17014	863	32	17
Flatpak	Inkscape	30731	17844	1517	39	22
Flatpak	LibreOffice	29562	18065	1594	44	23
AppImage	VLC	618	39	70	4	4
AppImage	GIMP	2117	168	231	0	0
AppImage	Inkscape	617	37	70	4	4
AppImage	LibreOffice	617	38	70	4	4

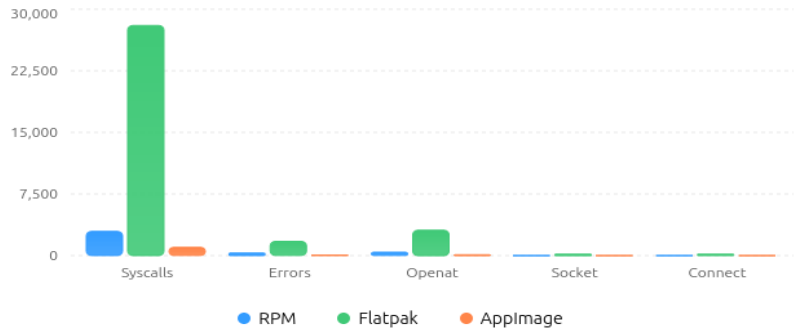


Рис. 3. Сравнение результатов strace-анализа приложений в форматах RPM, Flatpak и AppImage

Анализ журналов strace показал различия в количестве системных вызовов между исследуемыми форматами: наиболее высокие значения были

зафиксированы для Flatpak-приложений, поскольку перед запуском приложения дополнительно инициализируются runtime-компоненты, механизмы Bubblewrap, службы DBus и инфраструктура xdg-desktop-portal, в результате возрастает число обращений к файловой системе и межпроцессных взаимодействий. Для RPM-приложений подобные промежуточные компоненты отсутствуют, поэтому взаимодействие с операционной системой происходит более напрямую и сопровождается меньшим количеством системных вызовов. Наименьшие значения были получены для AppImage, что может объясняться отсутствием отдельной инфраструктуры контейнеризации и выполнением приложения как автономного исполняемого пакета.

Аудит доступа к файловой системе через lsof. Для анализа открытых файлов и библиотек использовалась утилита lsof (List Open Files) [12]. Приложение запускалось в обычном режиме, после чего выполнялась команда lsof -p PID, где PID соответствовал процессу исследуемого приложения. Для RPM использовались системные исполняемые файлы, для Flatpak – приложения, запущенные через flatpak run, а для AppImage – portable-сборки. В контексте исследования lsof позволяет определить, к каким файлам, каталогам, разделяемым библиотекам, сокетам X11/Wayland и сетевым соединениям обращается приложение во время работы, это необходимо для оценки потенциальных рисков безопасности (табл. 5)

Таблица 5. Результаты lsof-аудита приложений

Формат	APP	HOME	LIBS	X11	Wayland
RPM	VLC	1	77	0	0
RPM	GIMP	1	141	1	0
RPM	Inkscape	1	130	1	3
RPM	LibreOffice	1	12	1	0
Flatpak	VLC	16	180	3	3
Flatpak	GIMP	20	153	2	3
Flatpak	Inkscape	20	117	1	3
Flatpak	LibreOffice	25	98	2	3

AppImage	VLC	1	771	8	4
AppImage	GIMP	97	290	5	0
AppImage	Inkscape	19	145	1	3
AppImage	LibreOffice	N/A	N/A	N/A	N/A

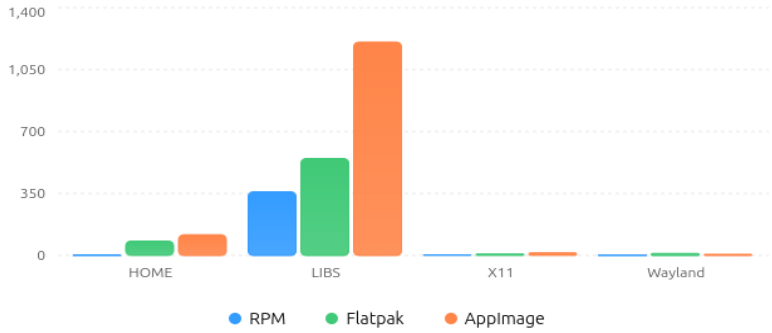


Рис. 4. Сравнение результатов lsof-аудита приложений в форматах RPM, Flatpak и AppImage

Результаты lsof-аудита продемонстрировали различия в характере доступа приложений к файловой системе и системным ресурсам в зависимости от используемого формата распространения. Наибольшие различия наблюдались по количеству загруженных библиотек (LIBS), что отражает особенности управления зависимостями в каждом формате. RPM-пакеты использовали преимущественно системные библиотеки, установленные в операционной системе, вследствие чего количество открытых файлов оставалось относительно небольшим. Flatpak-приложения продемонстрировали увеличение числа открытых библиотек за счет использования собственных runtime-компонентов, обеспечивающих независимость приложений от системного окружения. Наибольшее количество открытых библиотечных файлов было зафиксировано для AppImage. Это объясняется тем, что portable-пакет содержит внутри собственного образа значительную часть необходимых зависимостей и использует их независимо от системного набора библиотек.

Анализ обращений к каталогу HOME показал, что все исследованные приложения в той или иной степени взаимодействуют с пользовательскими

данными, однако характер такого взаимодействия различается: для Flatpak и AppImage число обращений оказалось выше, что, связано с созданием дополнительных конфигурационных файлов, кэша и служебных каталогов. Обращения к X11 и Wayland занимали сравнительно небольшую долю от общего числа открытых файлов. Судя по полученным данным, основная часть обращений возникает при подготовке среды выполнения и загрузке зависимостей.

Анализ сетевой активности. Для анализа сетевой активности использовалась команда strace -f -e trace=socket,connect <application>, позволяющая отслеживать системные вызовы создания соединений и сокетов. Полученные данные использовались для подсчета вызовов socket и connect, а также для разделения локальных Unix-сокетов (AF_UNIX) и сетевых соединений (AF_INET и AF_INET6). Такое разделение необходимо для корректной оценки реальной сетевой активности, поскольку Unix domain sockets отражают локальное межпроцессное взаимодействие внутри системы и не являются сетевыми TCP/IP-подключениями. Анализ выполнялся на этапе запуска приложений, что позволило исследовать сетевую активность, связанную с инициализацией программ и взаимодействием с компонентами операционной системы (табл. 6).

Таблица 6. Network trace: исходные total-значения и требуемое уточнение

Формат	APP	AF_UNIX	AF_INET/AF_INET6
RPM	VLC	16	0
RPM	GIMP	2	0
RPM	Inkscape	5	0
Flatpak	VLC	36	0
Flatpak	GIMP	44	0
Flatpak	Inkscape	48	0
AppImage	VLC	0	0
AppImage	GIMP	1	0

AppImage	Inkscape	15	0

Результаты анализа сетевой активности показали, что на этапе запуска исследуемых приложений не было зафиксировано реальных сетевых TCP/IP-соединений (AF_INET и AF_INET6) ни для одного из рассматриваемых форматов распространения. При этом наблюдалось различное количество вызовов socket и connect, однако их основная часть была связана с использованием локальных Unix-сокетов (AF_UNIX), обеспечивающих межпроцессное взаимодействие внутри операционной системы. Наибольшее число таких соединений было зафиксировано у Flatpak-приложений, что объясняется использованием дополнительной инфраструктуры изоляции, включая Bubblewrap, DBus и xdg-desktop-portal. RPM и AppImage продемонстрировали меньшее количество локальных соединений вследствие более прямой модели взаимодействия приложений с системой.

Permissions и sandbox-модель. Для RPM отдельной системы permissions уровня приложения нет, поэтому анализировались фактические обращения к ресурсам на основе результатов strace и lsof. Для Flatpak использовались команды flatpak info --show-permissions <application-id> и flatpak override --show, позволяющие определить набор разрешений приложения. Для AppImage дополнительно исследовались механизмы FUSE и пользовательских пространств имен с использованием команд ls -l /usr/bin/fusermount*, mount | grep fuse, groups и which bwrap (табл. 7).

Таблица 7. Permissions и sandbox-модель

Формат	Модель разрешений	Практический смысл
RPM	нет отдельной permissions-модели	контроль обеспечивается правами Linux-пользователя
Flatpak	network, ipc, X11/Wayland, filesystem, devices	доступ можно просматривать и изменять
AppImage	централизованной модели нет	зависит от host Linux, FUSE и прав пользователя

Проведенный анализ показал, что RPM-приложения выполняются с правами текущего пользователя и не используют собственных механизмов ограничения доступа. Flatpak позволяет отдельно управлять доступом к сети, файловой системе и устройствам, а также использует дополнительные механизмы изоляции, такие как Bubblewrap и система порталов. Благодаря этому поведение приложения становится более контролируемым. AppImage ориентирован прежде всего на переносимость и удобство распространения. Встроенной системы управления разрешениями в данном формате нет, поэтому уровень изоляции определяется настройками операционной системы и дополнительными средствами защиты.

Проверка целостности и общий hardening. Для RPM использовалась команда rpm --verify <package>, проверяющая соответствие установленных файлов данным пакетной базы. Для Flatpak применялись команды flatpak repair --dry-run и ostree fsck, выполняющие проверку целостности репозиториев без восстановления данных. Для AppImage контроль целостности осуществлялся посредством вычисления контрольных сумм командой sha256sum <appimage-file>.

Проверка показала, что каждый формат использует собственный подход к контролю состояния приложений. RPM позволяет выявлять изменения установленных файлов относительно данных пакетной базы и тем самым контролировать целостность системного программного обеспечения. Для Flatpak контроль распространяется не только на сами приложения, но и на используемые runtime-компоненты благодаря механизму OSTree. За счёт этого проверяется не отдельный файл, а вся среда выполнения приложения, включая общие зависимости. В AppImage контроль целостности обычно сводится к проверке контрольной суммы файла приложения. Такой подход позволяет убедиться, что загруженный образ не был изменен или поврежден, однако не предусматривает контроля зависимостей и состояния среды выполнения после запуска приложения

В целом результаты показали, что RPM и Flatpak предоставляют больше возможностей для контроля целостности программного обеспечения, тогда как AppImage ориентирован прежде всего на проверку неизменности самого файла приложения.

Установка, обновление, rollback, запуск без root, дисковое пространство (табл. 8)

Таблица 8. Результаты сопровождения

Критерий	RPM	Flatpak	AppImage
Установка	apt-get/rpm, обычно root	flatpak install, system/user	chmod +x и прямой запуск
Обновление	через системные репозитории	flatpak update	обычно вручную заменой файла
Rollback	downgrade/snapshot	OSTree/deployment модель	ручной выбор старого AppImage
Без root	обычно нет для системной установки	flatpak install --user	да, по умолчанию

Анализ использования дискового пространства показал, что наибольший объем данных занимают Flatpak-приложения, что обусловлено хранением runtime-сред и инфраструктуры OSTree (табл. 9). AppImage требует меньше места, однако включает собственные копии зависимостей внутри каждого portable-пакета, что приводит к дублированию компонентов при использовании нескольких приложений. RPM продемонстрировал наиболее эффективное использование дискового пространства за счет совместного использования системных библиотек и отсутствия необходимости хранения дополнительных сред выполнения.

Таблица 9. Дисковое пространство

Формат	Размер тестового набора	Интерпретация
RPM	1460.13 МБ	переиспользование shared libraries
Flatpak	4.9 ГБ	runtime и OSTree-хранилище
AppImage	3.7 ГБ	self-contained portable-модель

Ограничения исследования

Так как исследование выполнялось в виртуальной среде, абсолютные значения времени запуска, рендеринга и RSS могут отличаться на физическом

оборудовании, главное условие об одинаковой конфигурации всех виртуальных машин было соблюдено.

LibreOffice AppImage был исключен из итогового RSS-сравнения из-за невозможности стабильной фиксации целевого процесса в ходе эксперимента.

Часть Flatpak-трассировок выполнялась с sudo из-за особенностей bubblewrap и namespace-механизмов, что следует учитывать при интерпретации security-тестов.

Обсуждение результатов

Эксперимент показал, что наблюдаемые различия нельзя объяснить только особенностями выбранных приложений. Во многих случаях на результаты заметно влияли сами механизмы распространения и запуска, характерные для RPM, Flatpak и AppImage. Наиболее заметные различия наблюдаются в тестах времени запуска, потребления памяти и системной активности, где влияние дополнительных механизмов изоляции и автономности проявляется наиболее явно.

Полученные результаты в целом согласуются с наблюдениями, которые приводятся в опубликованных сравнительных работах по Flatpak и AppImage: в частности, отмеченное увеличение потребления памяти у Flatpak наблюдалось и в ранее опубликованных работах [8].

Следует учитывать, что измерение времени запуска выполнялось посредством запуска команд получения версии приложений с использованием утилиты Hyperfine. Такая методика позволяет оценить накладные расходы на инициализацию процесса и загрузку необходимых зависимостей, однако не отражает полное время загрузки графического интерфейса. Тем не менее одинаковые условия эксперимента для всех исследуемых форматов обеспечивают корректность сравнительного анализа и позволяют сопоставлять полученные результаты между собой.

Полученные результаты strace-анализа хорошо согласуются с архитектурными особенностями Flatpak. Как отмечают независимые исследователи, безопасность Flatpak обеспечивается за счет комбинации Linux

namespaces, seccomp-фильтрации и утилиты Bubblewrap [8] [9]. Именно эти механизмы генерируют дополнительный поток системных вызовов и обращений к локальным Unix-сокетам (AF_UNIX) на этапе инициализации, что объясняет максимальные значения syscalls, зафиксированные для Flatpak в таблице 4. Результаты эксперимента также подтверждают, что AppImage по умолчанию не использует встроенную песочницу и выполняется с правами текущего пользователя.

Несмотря на различия в количестве вызовов socket и connect, реальные TCP/IP-соединения в ходе эксперимента практически не наблюдались. Большинство зафиксированных обращений относилось к локальному межпроцессному взаимодействию через Unix-сокеты. По этой причине увеличение числа вызовов не следует напрямую связывать с сетевой активностью приложений. Для Flatpak значительная часть таких обращений обусловлена работой механизмов изоляции и взаимодействием с системными сервисами. Эксперимент выполнялся в виртуальной машине, поэтому абсолютные значения могут отличаться от результатов, полученных на физическом оборудовании.

Заключение

В рамках исследования был проведен сравнительный анализ форматов распространения программного обеспечения RPM, Flatpak и AppImage в среде ALT Workstation 10.4. Проведенные эксперименты показали, что RPM обеспечивает наиболее тесную интеграцию приложений с операционной системой и характеризуется минимальными накладными расходами по потреблению системных ресурсов. Flatpak продемонстрировал наиболее развитые механизмы изоляции, управления разрешениями и сопровождения приложений, однако использование runtime-компонентов и sandbox-инфраструктуры сопровождается увеличением объема используемых ресурсов. AppImage обеспечивает высокую переносимость и автономность приложений при отсутствии встроенных механизмов централизованного управления и изоляции.

Практическая значимость работы заключается в возможности использования полученных результатов при выборе подходящего формата распространения программного обеспечения для различных сценариев эксплуатации ALT Linux. Для корпоративных рабочих мест и систем с высокими требованиями к эффективности использования ресурсов наиболее целесообразным является применение RPM-пакетов. В случаях, когда приоритетом являются безопасность и изоляция приложений, предпочтительным решением является Flatpak. AppImage может быть рекомендован для распространения переносимых приложений и сценариев, не требующих интеграции с системой управления пакетами.

Результат проверки гипотез:

1. Первая гипотеза предполагала, что формат распространения программного обеспечения оказывает влияние на производительность запуска, потребление памяти и системную активность приложений. Результаты startup benchmark, memory benchmark и анализа системных вызовов показали наличие различий между RPM, Flatpak и AppImage. Следовательно, первая гипотеза подтверждается.

2. Вторая гипотеза предполагала, что Flatpak обеспечивает более высокий уровень изоляции и контроля доступа по сравнению с RPM и AppImage. Анализ разрешений приложений, системных вызовов, сетевой активности и механизмов проверки целостности показал наличие у Flatpak встроенной sandbox-модели и развитой системы управления permissions. Таким образом, вторая гипотеза также подтверждается.

3. Третья гипотеза была связана с предположением о том, что AppImage обеспечивает наибольшую автономность приложений. Полученные результаты в целом подтверждают данное предположение. AppImage действительно позволяет запускать приложения без установки и не зависит от пакетного менеджера, что делает его удобным для переносимого использования. Вместе с тем отсутствие встроенных механизмов централизованного

сопровождения и изоляции ограничивает возможности его применения в управляемой корпоративной среде.

В дальнейшем целесообразно провести аналогичные испытания для Astra Linux и РЕД ОС, а также дополнить исследование тестами, связанными с открытием документов, обработкой графики и длительной пользовательской нагрузкой.

Список использованных источников

1. Альт Рабочая станция 10.4 // Документация ALT Linux Team URL: https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html-single/alt-workstation/index.html (дата обращения: 17.05.2026).
2. Хабр // OSTree без магии: атомарные обновления, репозиторий и клиентский образ своими руками URL: https://habr.com/ru/articles/977760/ (дата обращения: 19.05.2026).
3. AppImage // ALT Linux Wiki URL: https://www.altlinux.org/Appimage (дата обращения: 19.05.2026).
4. AppImage // AppImage | Linux apps that run anywhere URL: https://appimage.org/ (дата обращения: 19.05.2026).
5. B.2.7. Verifying // Red Hat Documentation URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/deployment_guide/s2-rpm-verifying (дата обращения: 17.05.2026).
6. Basic concepts // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/basic-concepts.html (дата обращения: 18.05.2026).
7. Bubblewrap // Github URL: https://github.com/containers/bubblewrap (дата обращения: 19.05.2026).
8. Flatpak vs. Snap vs. AppImage - Linux Packaging Benchmarks! // TechHut URL: https://techhut.tv/flatpak-vs-snap-vs-appimage (дата обращения: 19.05.2026).

9. Flatpak vs Snap vs AppImage: Which Linux Package Format Should You Use? // Botmonster URL: https://botmonster.com/self-hosting/flatpak-vs-snap-vs-appimage-definitive-linux-packaging-comparison (дата обращения: 19.05.2026).

10. FUSE AppImage // Github URL: https://github.com/AppImage/AppImageKit/wiki/FUSE (дата обращения: 19.05.2026).

11. hyperfine // Github URL: https://github.com/sharkdp/hyperfine (дата обращения: 19.05.2026).

12. lsof(8) – Linux manual page // Michael Kerrisk - man7.org URL: https://man7.org/linux/man-pages/man8/lsof.8.html (дата обращения: 19.05.2026).

13. rpm (8) // rpm URL: https://rpm.org/docs/6.0.x/man/rpm.8 (дата обращения: 17.05.2026).

14. Sandbox Permissions // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/sandbox-permissions.html (дата обращения: 18.05.2026).

15. strace(1) – Linux manual page // Michael Kerrisk - man7.org URL: https://man7.org/linux/man-pages/man1/strace.1.html (дата обращения: 19.05.2026).

16. Traditional Packages vs Modern Formats: Which Linux App Installation Method Fits You Best in 2025 // Linux All Day URL: https://linuxallday.com/traditional-packages-vs-modern-formats-which-linux-app-installation-method-fits-you-best-in-2025/ (дата обращения: 16.05.2026).

17. Under the Hood // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/under-the-hood.html (дата обращения: 18.05.2026).

References

1. Alt Workstation 10.4 // Documentation of the ALT Linux Team URL: https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html-single/alt-workstation/index.html (accessed: 05/17/2026).

2. Habr // OSTree without magic: atomic updates, repository and client image with your own hands URL: https://habr.com/ru/articles/977760 / (accessed: 05/19/2026).

3. AppImage // AppImage | Linux apps that run anywhere URL: https://appimage.org / (accessed: 05/19/2026).

4. AppImage // ALT Linux Wiki URL: https://www.altlinux.org/Appimage (accessed: 05/19/2026).

5. B.2.7. Verifying // Red Hat Documentation URL: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/deployment_guide/s2-rpm-verifying (accessed: 05/17/2026).

6. Basic concepts // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/basic-concepts.html (accessed: 05/18/2026).

7. Bubblewrap // Github URL: https://github.com/containers/bubblewrap (accessed: 05/19/2026).

8. Flatpak vs. Snap vs. AppImage - Linux Packaging Benchmarks! // TechHut URL: https://techhut.tv/flatpak-vs-snap-vs-appimage/ (accessed: 05/19/2026).

9. Flatpak vs Snap vs AppImage: Definitive Linux Packaging Comparison // Botmonster URL: https://botmonster.com/self-hosting/flatpak-vs-snap-vs-appimage-definitive-linux-packaging-comparison (accessed: 05/19/2026).

10. FUSE AppImage // Github URL: https://github.com/AppImage/AppImageKit/wiki/FUSE (accessed: 05/19/2026).

11. hyperfine // Github URL: https://github.com/sharkdp/hyperfine (accessed: 05/19/2026).

12. lsof(8) – Linux manual page // Michael Kerrisk - man7.org URL: https://man7.org/linux/man-pages/man8/lsof.8.html (accessed: 05/19/2026).

13. rpm (8) // rpm URL: https://rpm.org/docs/6.0.x/man/rpm.8 (date of request: 05/17/2026).

14. Sandbox Permissions // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/sandbox-permissions.html (accessed: 05/18/2026).

15. strace(1) – Linux manual page // Michael Kerrisk - man7.org URL: https://man7.org/linux/man-pages/man1/strace .1.html (accessed: 05/19/2026).

16. Traditional Packages vs Modern Formats: Which Linux App Installation Method Fits You Best in 2025 // Linux All Day URL: https://linuxallday.com/traditional-packages-vs-modern-formats-which-linux-app-installation-method-fits-you-best-in-2025 / (accessed: 05/16/2026).

17. Under the Hood // Flatpak Documentation URL: https://docs.flatpak.org/en/latest/under-the-hood.html (accessed: 05/18/2026).

ДАННЫЕ ОБ АВТОРАХ

Бережная Полина Александровна, студентка 3 курса факультета комплексной безопасности топливно-энергетического комплекса

ФГАОУ ВО «РГУ нефти и газа (НИУ) имени И.М. Губкина», г. Москва, 119991, Российская Федерация

email: palinaberezhnaya047@gmail.com

Мирчевски Кристина Далиборовна, студентка 3 курса факультета комплексной безопасности топливно-энергетического комплекса

ФГАОУ ВО «РГУ нефти и газа (НИУ) имени И.М. Губкина», г. Москва, 119991, Российская Федерация

email: mirchevski23@gmail.com

DATA ABOUT THE AUTHORS

Berezhnaya Polina Aleksandrovna, 3rd year student, Faculty of Integrated Safety of the Fuel and Energy Complex

Gubkin Russian State University of Oil and Gas (National Research University), Moscow, 119991, Russian Federation

e-mail: palinaberezhnaya047@gmail.com

Mirchevski Kristina Daliborovna, 3rd year student, Faculty of Integrated Safety of the Fuel and Energy Complex

Gubkin Russian State University of Oil and Gas (National Research University), Moscow, 119991, Russian Federation

e-mail: mirchevski23@gmail.com